

```
"""Clean and analyse the GRIIS checklist
for India (Darwin Core Archive tables).
```

```
Input (same folder): raw_taxon.csv,
raw_speciesprofile.csv,
raw_distribution.csv
Output (same folder):
griis_india_clean.csv,
griis_india_accepted_invasive.csv,
                        chart_family.png,
chart_habitat.png, chart_kingdom.png
```

```
Usage: python clean_griis_data.py
"""
```

```
from pathlib import Path
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
```

```
HERE = Path(__file__).resolve().parent
```

```
def read(name: str) -> pd.DataFrame:
    # Read as text so identifiers keep
    their exact form.
    return pd.read_csv(HERE / name,
dtype=str)
```

```

def bar_chart(df, label, title, filename,
top=None):
    d = df.head(top) if top else df
    fig, ax = plt.subplots(figsize=(7, 0.45
* len(d) + 1.4))
    ax.barh(d[label][::-1], d["records"]
[:::-1], color="#2f6f4f")
    for y, v in enumerate(d["records"]
[:::-1]):
        ax.text(v + d["records"].max() *
0.01, y, str(v), va="center", fontsize=9)
    ax.set_title(title, loc="left",
fontsize=11, fontweight="bold")
    ax.set_xlabel("Accepted invasive
records")
    ax.spines[["top",
"right"]].set_visible(False)
    fig.tight_layout()
    fig.savefig(HERE / filename, dpi=200)
    plt.close(fig)

def main() -> None:
    taxon =
read("raw_taxon.csv")                #
Darwin Core Taxon core
    profile =
read("raw_speciesprofile.csv")        #
SpeciesProfile extension
    dist =
read("raw_distribution.csv")          #
Distribution extension

```

```

# 1. Validate: each table must have one
row per id.
for name, df in [("taxon", taxon),
("speciesprofile", profile),
("distribution", dist)]:
    assert df["id"].is_unique,
f"Duplicate ids in {name}"

# 2. Merge the core and both extensions
on id.
clean = (
    taxon.merge(profile, on="id",
how="left", validate="1:1")
    .merge(dist, on="id", how="left",
validate="1:1")
)
assert len(clean) == len(taxon), "Merge
changed the row count"

# 3. Clean: trim whitespace; recode
missing isInvasive.
#   GRIIS leaves isInvasive empty for
records that were not assessed,
#   so this means "Not evaluated", not
"non-invasive".
for col in clean.columns:
    clean[col] = clean[col].str.strip()
clean["isInvasive"] = (

clean["isInvasive"].replace({"Null": pd.NA,
"": pd.NA}).fillna("Not evaluated")
)
clean.to_csv(HERE /
"griis_india_clean.csv", index=False)

```

```

# 4. Analysis set: flagged invasive AND
accepted taxonomic name.
invasive = clean[clean["isInvasive"] ==
"Invasive"]
accepted =
invasive[invasive["taxonomicStatus"] ==
"ACCEPTED"]
accepted.to_csv(HERE /
"griis_india_accepted_invasive.csv",
index=False)

# 5. Summaries and charts.
def counts(col):
    out =
accepted[col].fillna("Unassigned").value_co
unts().rename_axis(col).reset_index(name="r
ecords")
    return out

bar_chart(counts("family"), "family",
"Top 10 families of accepted invasive
records (India)", "chart_family.png",
top=10)
bar_chart(counts("habitat"), "habitat",
"Habitat of accepted invasive records
(India)", "chart_habitat.png")
bar_chart(counts("kingdom"), "kingdom",
"Kingdom of accepted invasive records
(India)", "chart_kingdom.png")

print(f"Total records:
{len(clean)}")
print(f"Invasive (any status):

```

```
{len(invasive)}")
    print(f"Invasive + ACCEPTED:
{len(accepted)}")
    print(f"Not evaluated:
{(clean['isInvasive'] == 'Not
evaluated').sum()}")

if __name__ == "__main__":
    main()
```